

# *The Kernel & SDLC*

**מהווים חומר הרחבה.**



**השקפים המסומנים ב-**

**התוכן המובא בשקפים אלו אינו מהווה חלק מהחומר המחייב  
לבחינה בקורס.**

*The Kernel's code is:*  
*huge & extremely complex*

- בהרצאה הקודמת עסקנו בהבדל שבין המושגים 'מערכת הפעלה' ו- 'הגרעין'.
- ראינו כי מבחינה אקדמית והנדסית יש הבדל בין מושגים אלו.
- לא מדובר במושגים זהים.
- וציינו את העובדה שהקורס שלנו מתמקד בגרעין.
- בהרצאה הנוכחית נתחיל את ההתמקדות שלנו בגרעין.

## *The Scale and Complexity of Modern Operating System Codebases*

| Operating System       | Kernel Only (LoC)                           | Full System (LoC)                       |
|------------------------|---------------------------------------------|-----------------------------------------|
| Linux (e.g., 6.7)      | 30–35 million                               | >100 million (with distribution)        |
| Windows 11             | ~20 million (kernel core)                   | 50–70 million (full OS, estimates vary) |
| macOS (Ventura/Sonoma) | ~10–15 million (kernel and core components) | >80 million                             |

*Summary Table: Languages Used in Major Operating Systems*

| Operating System | Kernel Languages                      | Higher-Level System Languages    | Notes                                                         |
|------------------|---------------------------------------|----------------------------------|---------------------------------------------------------------|
| Linux            | C (95–98%), Assembly, Rust (emerging) | Shell, Python (build/test tools) | Rust is being evaluated for new safe subsystems               |
| Windows          | C, C++, Assembly                      | C#, PowerShell, .NET languages   | Extensive use of high-level languages in user-mode components |
| macOS            | C, C++, Assembly                      | Objective-C, Swift               | Swift is growing in system utilities and user-space daemons   |

- *Modern operating systems are among the most complex software systems ever engineered.*
- *In particular: the kernel*
- *As critical intermediaries between hardware and user-level applications, the kernel performs a vast array of services:*
  - *managing physical and virtual memory,*
  - *scheduling processes and threads,*
  - *handling I/O operations,*
  - *enforcing security policies,*
  - *abstracting file systems,*
  - *supporting networking protocols, and providing system-level APIs,*
  - *among many other responsibilities.*

***It is crucial to design and implement the kernel of a modern OS using a methodical and structured SDLC approach !!***

*- It is not only necessary but crucial to design and implement a modern operating system (OS), and particularly the kernel, using a methodical and structured approach, adhering to the principles of the Software Development Life Cycle (SDLC).*

*- This necessity stems from a combination of technical, managerial, and practical reasons. Below is a comprehensive and detailed explanation that addresses the multifaceted rationale behind this assertion*



*Why SDLC Is Crucial ?  
Justifications*

## ***Justification #1: High system complexity necessitates structured planning***

- *Modern operating systems such as Windows, macOS, and Linux contain millions of lines of code, modular components, device drivers, process schedulers, memory managers, and security subsystems.*
- *Designing such systems informally or ad hoc increases the risk of:*
  - *Architectural inconsistencies*
  - *Integration failures*
  - *Non-functional requirement violations (e.g., performance, scalability)*
- *Explanation:*
  - *Without a formal SDLC, the system would likely suffer from chaotic evolution.*
  - *SDLC enforces phased development (requirements, design, implementation, testing, deployment, maintenance), which ensures that complex interdependencies are managed predictably and reliably.*

## ***Justification #2: Reliability and Stability Requirements***

- *The kernel must ensure uptime, consistency, and resilience.*
- *A kernel is the foundational layer upon which all application software depends.*
- *Any failure at the kernel's level can cascade into system-wide crashes, data loss, or security breaches.*
  
- *Explanation:*
  - *The SDLC includes systematic verification and validation phases.*
  - *Design reviews, code audits, unit tests, integration tests, and system tests help discover and eliminate bugs early.*
  - *Without this rigor, reliability and stability cannot be guaranteed.*

### ***Justification #3: Safety and Security Constraints***

- *The kernel manages critical resources and must guard against threats.*
- *Modern kernels handle privileged operations, memory protection, file system integrity, and enforce access control policies.*
- *They are frequent targets of malware and cyberattacks.*
  
- *Explanation:*
- *Security considerations must be integrated from the early stages (requirements and design), following practices like secure design patterns, threat modeling, and code hardening.*
- *The SDLC provides a framework to integrate security throughout development (commonly known as Security Development Lifecycle).*

## ***Justification #4: Scalability and Extensibility Needs***

- *The kernel evolves to support new hardware and software paradigms.*
- *As technology advances, kernels must support new CPU architectures, storage technologies, and paradigms like virtualization and containerization.*
- *Explanation:*
  - *Structured design (as enabled by SDLC) allows developers to use modular architecture and clean interface definitions (e.g., APIs, kernel modules).*
  - *This facilitates future extensions without having to overhaul the entire system.*

## ***Justification #5: Team Collaboration and Project Management***

- *The development of OSs and their kernels is a large-scale collaborative effort.*
- *Modern OS projects often involve hundreds or thousands of contributors, including cross-functional teams (kernel developers, UI/UX, QA, documentation).*
- *Explanation:*
  - *The SDLC imposes discipline through documentation, version control, milestone planning, and communication standards.*
  - *It enables coordination across diverse teams and helps stakeholders manage the project efficiently.*

## ***Justification #6: Cost and Time Efficiency***

- *Structured approaches reduce rework and technical debt.*
- *Ad hoc or poorly managed development often leads to:*
  - *Late discovery of flaws*
  - *Costly re-engineering*
  - *Slipped deadlines*
- *Explanation:*
  - *SDLC techniques such as requirement traceability, iterative development (e.g., Agile or Spiral models), and risk management reduce uncertainty and enable better resource allocation.*

## ***Justification #7: Maintainability and Lifecycle Support***

- *OSs and their kernels require long-term maintenance and updates.*
- *An operating system's lifecycle spans decades.*
- *For example, UNIX systems have evolved for over 50 years.*
  
- *Explanation:*
  - *Adherence to SDLC practices like modular design, coding standards, documentation, and change control ensures maintainability.*
  - *It simplifies debugging, updates, and user support over time.*



## ***Justification #8: Legal and Regulatory Compliance***

- *Certain OS domains (e.g., medical, aviation) require certification.*
- *In regulated industries, operating systems and their kernel must comply with standards such as:*
  - *ISO/IEC 12207 (software lifecycle processes)*
  - *DO-178C (aviation)*
  - *IEC 62304 (medical software)*

*Explanation: Compliance requires traceable processes, rigorous testing, and documentation — all inherent in a structured SDLC.*

## ***Justification #9: User and Developer Experience***

- *Well-engineered OSs and their kernels improve usability and foster ecosystems.*
- *Developers depend on clear documentation, APIs, and predictable behavior to build applications.*
- *Users expect consistency and responsiveness.*
  
- *Explanation:*
  - *A disciplined development process ensures consistent user interfaces, developer tools, and robust error handling, enhancing the overall experience.*

## **Conclusion**

- *Considering all the above factors, it is unequivocally necessary to design and implement modern OSs and their kernels through a methodical, structured, and disciplined approach as provided by the Software Development Life Cycle.*
- *The SDLC provides the framework, processes, and checkpoints that mitigate risk, ensure quality, and enable sustainable development in the face of growing technical and organizational complexity.*

# *Requirements & Design Goals*

## שלב איסוף, ניתוח וגיבוש דרישות

- זיהוי בעלי עניין: מפתחי קרנל, מנהלי Distribution, מתכנתי Application-level, ומפעילי שרתים.

- דרישות פונקציונליות

- 'שיוך' כל דרישה לבעל עניין. אחד לפחות.

- לדוגמה, תמיכה במערכת קבצים חדשה או פרוטוקול רשת.

- דרישות לא פונקציונליות

- קריטריות במיוחד – זמן תגובה (Latency) של ה-Interrupts, עמידות ב-Load גבוה, ואיכות ה-

Security.

- התוצר: מסמך ה- SRS (Software Requirements Specification) המהווה את ה-Blueprint המוסכם על הקהילה.

## גיבוש רשימת ה- *system calls* שה- *kernel* 'מספק'/תומך בהן

- גרסה 1.0

- קריאת מעכת ב'השראת *Unix*'.

- גרסאות הבאות: הכרח *{!!!}* לשמור על תאימות לאחור [= *backward comparability*].

- רשימת קריאות המערכת הנתמכות בגרסה 6.14:

- <https://filippo.io/linux-syscall-table/>

## *Examples of System Calls*

### Process Control

`fork()`  
`exit()`  
`wait()`

### File Manipulation

`open()`  
`read()`  
`write()`  
`close()`

### Device Manipulation

`ioctl()`  
`read()`  
`write()`

### Information Maintenance

`getpid()`  
`alarm()`  
`sleep()`

### Communication

`pipe()`  
`shmget()`  
`mmap()`

### Protection

`chmod()`  
`umask()`  
`chown()`

### EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

|                                      |                                                  |            |
|--------------------------------------|--------------------------------------------------|------------|
| <pre>#include &lt;unistd.h&gt;</pre> |                                                  |            |
| <pre>ssize_t</pre>                   | <pre>read(int fd, void *buf, size_t count)</pre> |            |
|                                      |                                                  |            |
| return                               | function                                         | parameters |
| value                                | name                                             |            |

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

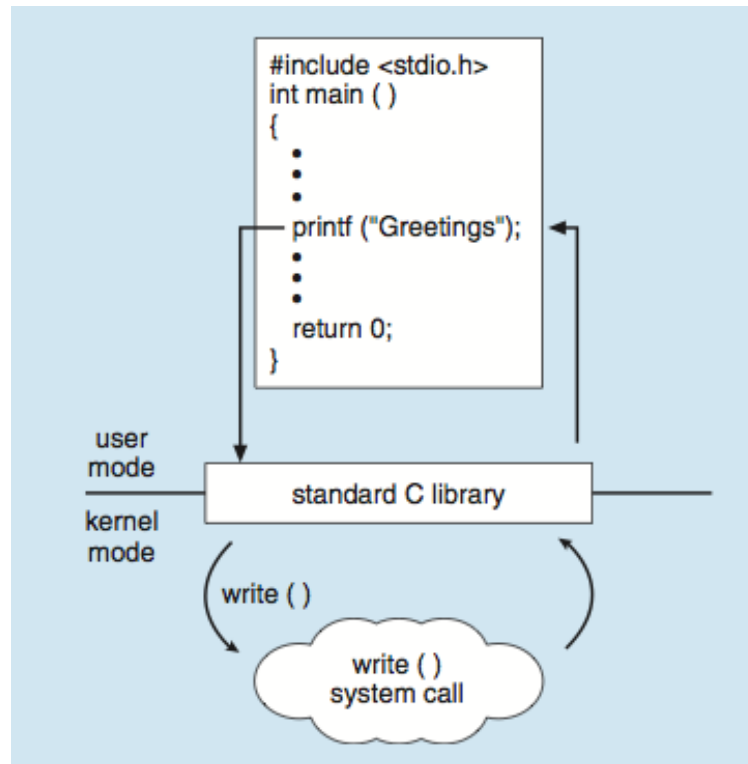
- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.



## System Calls & the glibc Library

- C program invoking printf() library call, which calls write()



The primary function of the wrapper is to

- Place all the **arguments** to be passed to the system call in the appropriate processor registers (& maybe on the call stack as well)
- Set a unique **system call number** for the kernel to call. In this way the library, which exists between the OS and the application, increases portability.

# *Linux Requirements & Design Goals: Elaboration*

## *Linux design goals*

- *Linux, an open-source Unix-like operating system, has been designed with several key goals that guide its development, usage, and evolution.*
- *These goals encompass aspects of performance, security, flexibility, and user freedom. Below is an in-depth examination of these design goals.*

## ***1. Performance and Efficiency***

- *Linux aims to be highly performant and efficient, allowing it to run on a wide range of hardware from supercomputers to embedded systems.*
- *Key aspects include:*
  - *Kernel Optimization:*
    - *The Linux kernel is optimized for speed and minimal resource usage.*
    - *It supports efficient process scheduling, memory management, and I/O operations.*
  - *Modularity:*
    - *The kernel's modular design allows loading only necessary components, reducing overhead and improving performance.*
  - *Scalability:*
    - *Linux is designed to scale effectively from small embedded devices with limited resources to large multi-core servers and supercomputers, maintaining performance across diverse environments.*

## 2. Security

- *Security is a paramount goal in Linux development, aiming to provide a secure operating environment for all types of applications and users.*
- *User and Group Permissions:*
  - *Linux implements a robust permissions system to control access to files and resources.*
- *Security Modules:*
  - *Features like SELinux (Security-Enhanced Linux) provide advanced security policies and mandatory access control (MAC).*
- *Regular Updates:*
  - *Continuous updates and patches ensure vulnerabilities are addressed promptly.*
- *Isolation:*
  - *Technologies like namespaces and cgroups provide process and resource isolation, enhancing security.*

### ***3. Stability and Reliability***

- *Stability and reliability are crucial for Linux, especially in enterprise and critical systems.*
- *Robust Kernel:*
  - *The Linux kernel is designed to handle high loads and uptime with minimal crashes.*
- *Long-Term Support (LTS):*
  - *LTS versions of Linux distributions receive extended maintenance, ensuring stability for several years.*
- *Testing and Validation:*
  - *Extensive testing and validation processes, including community and automated testing, ensure high reliability.*

## ***4. Flexibility and Customization***

*Linux's design allows extensive customization to meet diverse user and application needs.*

- Open Source:*

- As an open-source project, Linux can be modified and tailored to specific requirements.*

- Configurable Kernel:*

- Users can configure and compile the kernel to include only needed features and drivers.*

- Diverse Distributions:*

- The variety of Linux distributions (distros) provides tailored environments for different use cases, from desktops to servers to embedded systems.*

## **5. Portability**

*Linux is designed to be portable across various hardware architectures.*

- *Architecture Support:*

- *The kernel supports numerous architectures, including x86, ARM, PowerPC, and more.*

- *Cross-Compilation:*

- *Tools and frameworks support cross-compilation, enabling development for different hardware platforms from a single environment.*

- *Device Drivers:*

- *A vast library of device drivers ensures compatibility with a wide range of hardware components.*



## **6. Interoperability**

*Interoperability with other systems and standards is a key consideration in Linux design.*

- *POSIX Compliance:*

- *Linux adheres to POSIX (Portable Operating System Interface) standards, ensuring compatibility with other Unix-like systems.*

- *File System Support:*

- *Linux supports a variety of file systems, including ext4, NTFS, FAT32, and more, facilitating data exchange with other operating systems.*

- *Network Protocols:*

- *Comprehensive support for standard network protocols ensures smooth communication in mixed-OS environments.*

## ***7. Community and Collaboration***

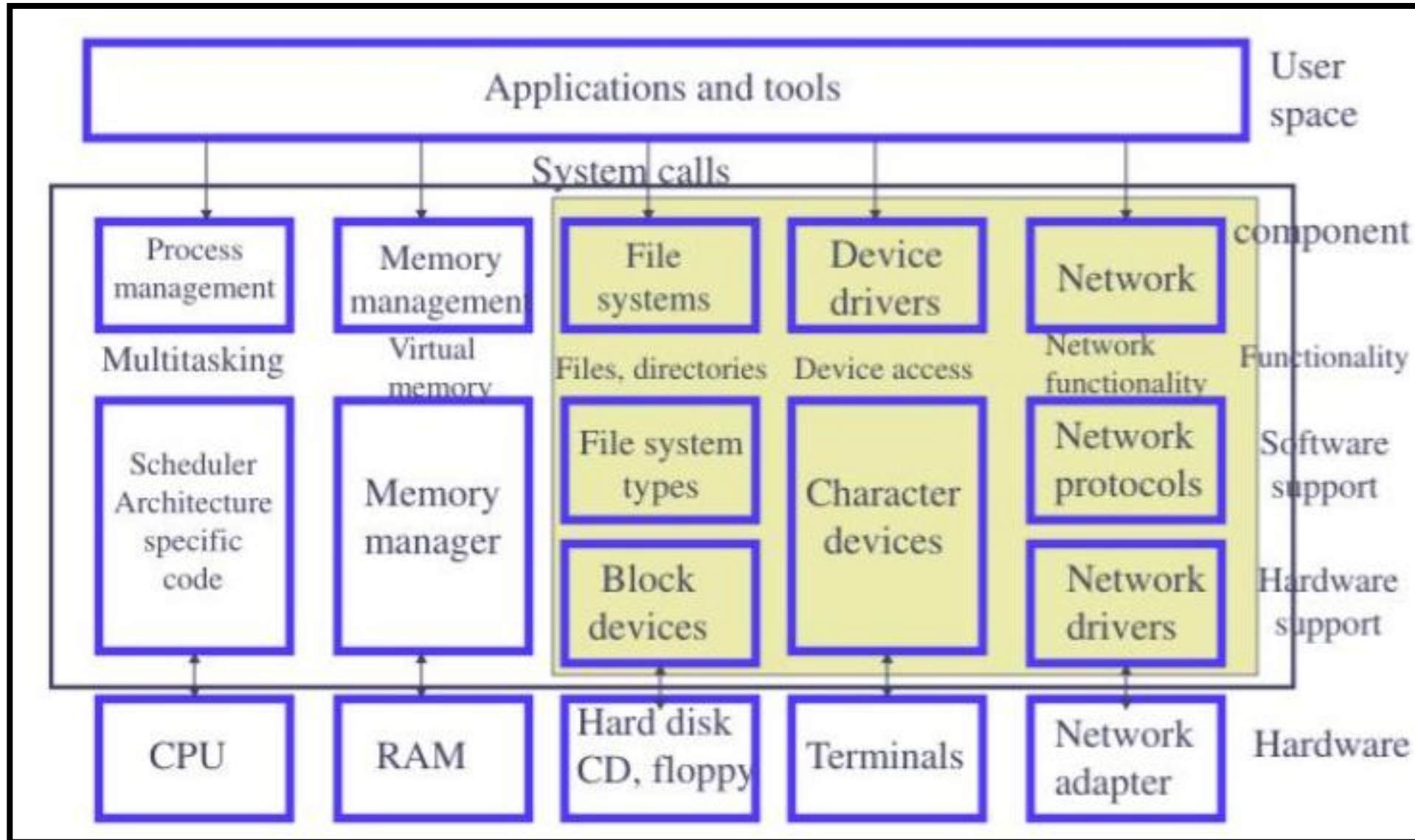
- *The Linux development model emphasizes community collaboration and open-source principles.*
- *Open Development:*
  - *Contributions from a global community of developers enhance the robustness and innovation of the Linux kernel.*
- *Transparent Processes:*
  - *Development processes, including mailing lists, bug tracking, and version control, are open to public scrutiny.*
- *Diversity of Contributions: Contributions come from individuals, academia, and major corporations, fostering a diverse and rich development ecosystem.*

## ***8. Minimalism and Elegance***

- *Linux aims for simplicity and minimalism in its design.*
- *KISS Principle:*
  - *The "Keep It Simple, Stupid" principle ensures the system remains straightforward and avoid unnecessary complexity.*
- *Unix Philosophy:*
  - *Adherence to the Unix philosophy of building simple, modular, and reusable components that perform specific tasks well.*

*Design:*  
*HLD & LLD*

## Structure of Linux Kernel



***Kernel's modules***

## ***Kernel's modules [components]***

- *Process Management*
  - *Creation/deletion of processes*
  - *processes Suspension/resumption*
  - *Mechanism for communication*
  - *Deadlock handling*
- *Main Memory Management*
  - *Keep track of used and available memory segments*
  - *Decision which segment will be allocated to a given process*
  - *Allocate and deallocate memory space*
- *File Management*
  - *Creation and deletion of files*
  - *Creation/deletion of directories*
  - *Support manipulation file and directories primitives*
  - *Mapping files into secondary storage*

## ***Kernel's modules [components]***

- *I/O System Management*
  - *Memory management component including buffering, caching, and spooling*
  - *General device-driver interface*
  - *Drivers for specific hardware devices*
- *Secondary-storage management*
  - *Free space management*
  - *Storage allocation*
  - *Disk scheduling*
- *Networking*
  - *TCP/IP protocols*
  - *Memory management component including buffering, caching, and spooling*



## ***Kernel's modules [components]***

- *Protection system*
  - *Process can act only on those that they are authorized of*
  - *Protection of process memory address space from other process*
- *Error and faults handling*
- *Detection and handling of resources lost or waste*
- *Detection and handling of over utilization resources (denial of service attacks)*

כותרת

תוכן

כותרת

תוכן

כותרת

תוכן